

150. Evaluate Reverse Polish Notation Medium

Acceptance: 56.7% | [LeetCode](#)

You are given an array of strings `tokens` that represents an arithmetic expression in a [Reverse Polish Notation](#).

Evaluate the expression. Return *an integer that represents the value of the expression*.

Note that:

- The valid operators are `'+'`, `'-'`, `'*'`, and `'/'`.
- Each operand may be an integer or another expression.
- The division between two integers always **truncates toward zero**.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.
- The answer and all the intermediate calculations can be represented in a **32-bit** integer.

Example 1:

```
**Input:** tokens = ["2","1","+","3","*"]
**Output:** 9
**Explanation:** ((2 + 1) * 3) = 9
```

Example 2:

```
**Input:** tokens = ["4","13","5","/","+"]
**Output:** 6
**Explanation:** (4 + (13 / 5)) = 6
```

Example 3:

```
**Input:** tokens =
["10","6","9","3","+","-11","*","/", "*", "17","+","5","+" ]
**Output:** 22
**Explanation:** ((10 * (6 / ((9 + 3) * -11))) + 17) +
5
= ((10 * (6 / (12 * -11))) + 17) + 5
= ((10 * (6 / -132)) + 17) + 5
= ((10 * 0) + 17) + 5
= (0 + 17) + 5
= 17 + 5
= 22
```

Constraints:

- `1 <= tokens.length <= 104`
- `tokens[i]` is either an operator: `"+"`, `"-"`, `"*"`, or `"/"`, or an integer in the range `[-200, 200]`.

Claude Opus Solution

Evaluate Reverse Polish Notation — Solution

1. Intuition

Reverse Polish Notation (RPN), also called *postfix notation*, places operators *after* their operands. For example, `3 + 4` becomes `3 4 +`.

The key insight: **RPN is naturally evaluated using a stack**.

- When we see a **number**, we don't know what to do with it yet, so we **push it onto a stack** (defer its use).
- When we see an **operator**, both its operands are the two most recently seen values — i.e., the **top two elements of the stack**. We pop them, apply the operator, and push the result back.

This works because RPN has no parentheses — the structure of the expression is fully determined by operand/operator order, making the stack the perfect data structure (Last-In-First-Out matches how operands attach to operators).

Critical detail — operand order: Since we pop in reverse, the *first pop is the right operand* and the *second pop is the left operand*. This matters for non-commutative operators `-` and `/`.

2. Approach (Pseudocode)

```
initialize empty stack

for each token in tokens:
    if token is an operator (+, -, *, /):
        right = pop from stack    # popped FIRST → right operand
        left = pop from stack    # popped SECOND → left operand
        result = apply operator on (left, right)
        push result onto stack
    else:
        # token is a number
        push int(token) onto stack

return top of stack          # final remaining value is the answer
```

Watch out for division: The problem says division **truncates toward zero** (e.g., $-7 / 2 = -3$, not -4). Python's `//` floors toward negative infinity, so we must use `int(a / b)` or `int(a // b)` instead.

3. Python Solution

```
from typing import List

class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        stack = []

        # Map operators to functions. Using lambdas keeps the main loop clean.
        # NOTE: each lambda takes (left, right) in correct operand order.
        operations = {
            "+": lambda l, r: l + r,
            "-": lambda l, r: l - r,
            "*": lambda l, r: l * r,
            # int(l / r) truncates TOWARD ZERO (required by the problem).
            # We can't use l // r because Python's floor division rounds
            # toward negative infinity (e.g. -7 // 2 == -4, but we need -3).
            "/": lambda l, r: int(l / r),
        }

        for token in tokens:
            if token in operations:
                # Operator found -> its two operands are on top of the stack.
                # The first pop is the RIGHT operand (was pushed last),
                # the second pop is the LEFT operand.
                right = stack.pop()
                left = stack.pop()

                # Apply the operator and push the result back.
                # The result acts as an operand for any future operator.
                stack.append(operations[token](left, right))
            else:
                # Token is a number -> convert to int and defer by pushing.
                stack.append(int(token))

        # After processing all tokens, exactly one value remains: the answer.
        return stack[-1]
```

4. Complexity Analysis

Time Complexity: $O(n)$ where $n = \text{len}(\text{tokens})$. - We iterate through every token exactly once. - Each operation (push, pop, dictionary lookup, arithmetic) is $O(1)$.

Space Complexity: $O(n)$. - In the worst case (e.g., all numbers come first like `["1", "2", "3", ..., "+", "+", ...]`), the stack holds up to $\sim n/2$ elements simultaneously, which is $O(n)$.

5. Alternative Approaches

a) Recursion (process from the back)

RPN can be parsed recursively by reading from the **end**: - The last token is the root operator. - Recursively evaluate the right subtree, then the left subtree (consuming tokens backward).

This builds an implicit expression tree but uses $O(n)$ call-stack space and is harder to reason about than the iterative stack. Generally not preferred.

b) Convert to Expression Tree, then evaluate

Build an actual binary tree where leaves are numbers and internal nodes are operators, then do a post-order traversal. This is conceptually clean and useful if you need the tree for other purposes (e.g., converting back to infix), but it's overkill here — same complexity with extra overhead.

c) Reusing the input array as a stack

For a tiny space optimization, you can reuse the `tokens` list itself with an index pointer to simulate the stack, avoiding a separate list. This still has $O(n)$ space (the input) but no *extra* auxiliary structure. Rarely worth the reduced readability.

Key Takeaways

Concept	Why it matters
Stack for postfix	LIFO order perfectly matches how operands attach to operators in RPN.
Operand order on pop	First pop = right operand, second pop = left. Crucial for <code>-</code> and <code>/</code> .
Truncation toward zero	Use <code>int(l / r)</code> , not <code>l // r</code> , to match the problem's division rule.

This is a classic **stack** pattern that also applies to: valid parentheses, infix-to-postfix conversion, and basic calculator problems.

Codex Solution

1. Intuition

Reverse Polish Notation, or postfix notation, places the operator **after** its operands.

For example:

```
["2", "1", "+", "3", "*"]
```

means:

```
(2 + 1) * 3
```

The key pattern is:

When we see a number, save it.

When we see an operator, apply it to the two most recent saved numbers.

This is exactly what a **stack** is good for.

A stack follows **Last In, First Out** behavior, which matches Reverse Polish Notation because the most recent operands are the ones used by the next operator.

2. Approach

Use a stack to store operands.

For each token:

1. If the token is a number:
2. Convert it to an integer.
3. Push it onto the stack.
4. If the token is an operator:
5. Pop the right operand from the stack.
6. Pop the left operand from the stack.
7. Apply the operator.
8. Push the result back onto the stack.
9. At the end, the stack contains exactly one value:
10. This is the final result.

Important detail:

For subtraction and division, operand order matters.

If the stack has:

```
[4, 13, 5]
```

and the operator is `/`, we do:

```
13 / 5
```

not:

```
5 / 13
```

So we must pop in this order:

```
right = stack.pop()
left = stack.pop()
```

Pseudocode

```
initialize empty stack

for token in tokens:
    if token is an operator:
        right = pop from stack
        left = pop from stack

        if token is "+":
            result = left + right
        else if token is "-":
            result = left - right
        else if token is "*":
            result = left * right
        else if token is "/":
            result = division of left by right truncated toward zero

        push result onto stack

    else:
        convert token to integer
        push it onto stack

return top of stack
```

3. Python Solution

```
from typing import List

class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        # Stack stores operands and intermediate results.
        stack = []

        # Set of valid operators for quick lookup.
        operators = {"+", "-", "*", "/"}

        for token in tokens:
            # If the current token is an operator, we must apply it
            # to the two most recent operands.
            if token in operators:
                # The first popped value is the right operand.
                right = stack.pop()

                # The second popped value is the left operand.
                left = stack.pop()

                # Apply the operator.
                if token == "+":
                    result = left + right
                elif token == "-":
                    result = left - right
                elif token == "*":
                    result = left * right
```

```

else:
    # Division must truncate toward zero.
    #
    # Python's // operator floors toward negative infinity:
    # Example: -3 // 2 == -2
    #
    # But the problem wants truncation toward zero:
    # Example: -3 / 2 should become -1
    #
    # int(left / right) works because int() truncates toward zero.
    result = int(left / right)

    # Push the intermediate result back onto the stack.
    stack.append(result)

else:
    # The token is a number.
    # Convert it from string to integer and push it.
    #
    # This correctly handles negative numbers like "-11",
    # because "-11" is not in the operators set.
    stack.append(int(token))

# After processing all tokens, the stack contains one value:
# the final result of the expression.
return stack[-1]

```

4. Complexity Analysis

Let n be the number of tokens.

Time Complexity: $O(n)$

We process each token exactly once.

For every token, stack operations such as `append` and `pop` take $O(1)$ time.

Therefore, total time complexity is:

$O(n)$

Space Complexity: $O(n)$

In the worst case, many operands may be pushed before any operators appear.

Example:

```
["1", "2", "3", "4", "+", ...]
```

So the stack can grow up to $O(n)$ size.

Therefore, space complexity is:

$O(n)$

5. Alternative Approaches

Alternative 1: In-place Stack Using the Input Array

Instead of creating a separate stack, we could reuse the `tokens` array as a stack by overwriting values.

However, this makes the code less clean and is usually unnecessary unless memory optimization is required.

Space complexity would still be $O(n)$ in terms of input storage, but auxiliary space could be reduced.

Alternative 2: Recursive Evaluation from Right to Left

Since Reverse Polish Notation places operators after operands, we can evaluate recursively by scanning from the end.

For example:

```
["2", "1", "+", "3", "*"]
```

The last token is `"*"`, so recursively evaluate its right and left subexpressions.
However, this approach is more complicated and may use recursion stack space.
The iterative stack solution is simpler, safer, and standard for this problem.